

Strategy

1. Explicación del patrón

Strategy es un patrón de comportamiento que permite adaptar el comportamiento o los algoritmos de un objeto según una necesidad concreta, sin cambiar la forma en la que ese objeto interactúa con sus clientes.

La idea principal es separar los distintos algoritmos en clases independientes, llamadas estrategias, y hacer que el objeto principal delegue en una de esas estrategias.

De esta forma, el objeto principal no implementa directamente todas las variantes posibles de un comportamiento. En su lugar, mantiene una referencia a una estrategia y le pide que realice el trabajo.

Esto permite cambiar el algoritmo utilizado sin modificar el código del objeto principal ni el código cliente que lo utiliza.

2. Cuándo se utiliza

Se utiliza el patrón Strategy cuando:

- Una clase puede realizar un mismo comportamiento de varias formas distintas.
- Existen varios algoritmos posibles para resolver una tarea.
- Algunos algoritmos son más eficaces en tiempo de ejecución o en consumo de memoria.
- La elección del algoritmo depende de una necesidad concreta.
- El código tiene muchas instrucciones condicionales para seleccionar qué algoritmo ejecutar.
- Hay muchas clases casi iguales que solo se diferencian en una parte de su comportamiento.
- Se quiere cambiar el comportamiento de un objeto durante la ejecución.
- Se quiere evitar que una clase principal crezca demasiado al añadir nuevas variantes de un algoritmo.
- Se quiere aislar la lógica de negocio de los detalles concretos de implementación de los algoritmos.

3. Cómo aplicarlo

Para aplicar Strategy se suelen definir estos participantes:

Estrategia

Es la interfaz común a todos los algoritmos.

Define el método que deberán implementar todas las estrategias concretas.

Ejemplo:

- `calcula()`
- `sort(array)`
- `execute(a, b)`

La entidad o contexto usa esta interfaz para invocar el algoritmo sin depender de una clase concreta.

Estrategia concreta

Es una implementación concreta de la interfaz estrategia.

Cada estrategia concreta representa una variante distinta del algoritmo.

Ejemplos:

- `EstrategiaConcretaA`
- `EstrategiaConcretaB`
- `EnvioEstandar`
- `EnvioExpress`
- `EnvioNocturno`
- `BubbleSort`
- `SelectionSort`
- `Suma`
- `Resta`
- `Multiplicacion`
- `Division`

Todas implementan la misma interfaz, pero cada una resuelve el comportamiento de una forma diferente.

Entidad o contexto

Es la clase que necesita usar uno de los algoritmos.

Responsabilidades:

- Mantener una referencia a una estrategia.
- Delegar en la estrategia la ejecución del algoritmo.
- Permitir, si es necesario, cambiar la estrategia utilizada.
- Mantener la misma interfaz de cara al cliente aunque cambie el algoritmo interno.

Ejemplo:

La entidad puede tener un método como `solicita()`, y dentro de ese método llamar a:

```
estrategia.calcula()
```

El cliente sigue usando la entidad igual que antes, pero el comportamiento real depende de la estrategia asociada.

Cliente

El cliente normalmente elige qué estrategia concreta se va a usar y se la pasa a la entidad.

Después, el cliente trabaja con la entidad, no directamente con los detalles del algoritmo.

4. Forma práctica de aplicarlo

Pasos:

1. Detectar una clase que tenga varios comportamientos o algoritmos alternativos.
2. Localizar si esos comportamientos están implementados mediante muchos `if`, `switch` o condicionales.
3. Crear una interfaz común para representar el algoritmo.
4. Extraer cada variante del algoritmo a una clase concreta.
5. Hacer que todas las estrategias concretas implementen la misma interfaz.
6. Añadir en la entidad un atributo que guarde una referencia a la estrategia.
7. Hacer que la entidad delegue el trabajo en la estrategia.
8. Permitir que el cliente indique qué estrategia quiere usar.
9. Si hace falta cambiar el comportamiento en tiempo de ejecución, añadir un método para cambiar la estrategia.
10. Para añadir nuevos algoritmos, crear nuevas estrategias concretas sin modificar la entidad.

5. Idea clave

El objeto principal no debe contener todos los algoritmos posibles.

Debe delegar el comportamiento variable en una estrategia.

Así, el algoritmo puede cambiar sin cambiar la forma en la que el cliente usa el objeto.

6. Ejemplos del material

Ejemplo de envíos

Una clase `Envio` puede usar distintas estrategias de envío.

Estrategias posibles:

- `EnvioEstandar`
- `EnvioExpress`
- `EnvioNocturno`

Todas implementan una interfaz común, por ejemplo `EstrategiaEnvio`.

La clase `Envio` no necesita conocer los detalles de cada tipo de envío. Solo usa la estrategia seleccionada.

Ejemplo de ordenación

Una clase `SortEntidad` puede delegar el algoritmo de ordenación en una estrategia.

Interfaz común:

- `SortStrategy`

Estrategias concretas:

- BubbleSort
- SelectionSort

La entidad tiene una referencia a `SortStrategy` y puede cambiar de estrategia en tiempo de ejecución.

Esto permite usar distintos algoritmos de ordenación sin modificar la clase principal.

Ejemplo de calculadora

Una calculadora puede usar distintas operaciones como estrategias.

Interfaz común:

- Operacion

Estrategias concretas:

- Suma
- Resta
- Multiplicacion
- Division

La calculadora delega la operación en la estrategia seleccionada.

7. Ventajas prácticas

- Permite intercambiar algoritmos durante la ejecución.
- Evita condicionales grandes para elegir entre algoritmos.
- Separa el algoritmo del objeto que lo utiliza.
- Permite añadir nuevas estrategias sin modificar la entidad principal.
- Reduce la duplicación cuando varias clases solo se diferencian en un comportamiento.
- Hace que el código sea más flexible y fácil de mantener.
- Permite que el cliente seleccione el comportamiento que necesita.

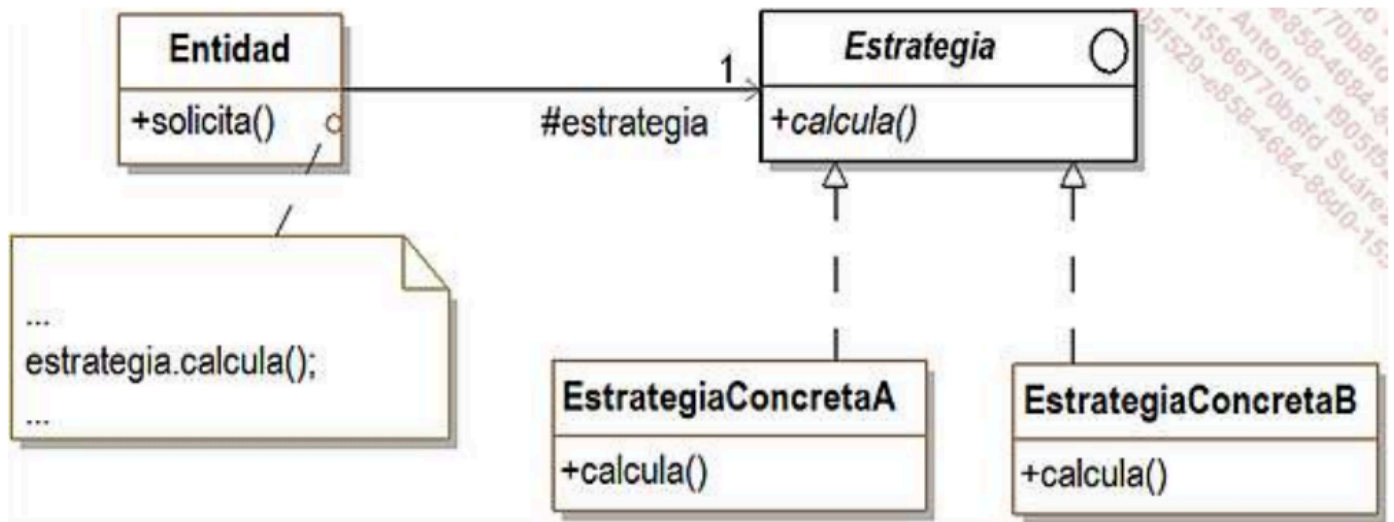
8. Cuidado

No siempre merece la pena aplicar Strategy.

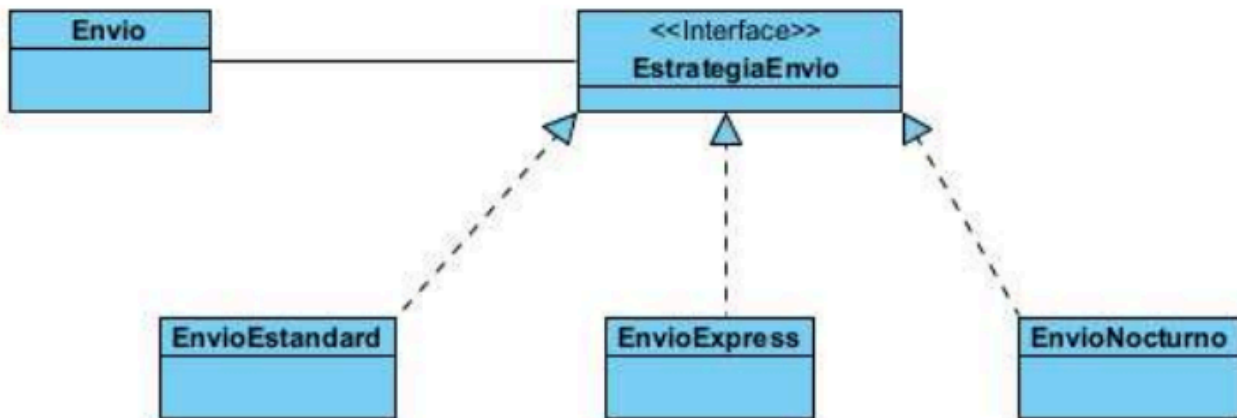
Si solo existen uno o dos algoritmos simples y apenas van a cambiar, crear muchas clases e interfaces puede complicar innecesariamente el diseño.

También hay que tener en cuenta que el cliente debe conocer las diferencias entre estrategias para poder elegir la adecuada.

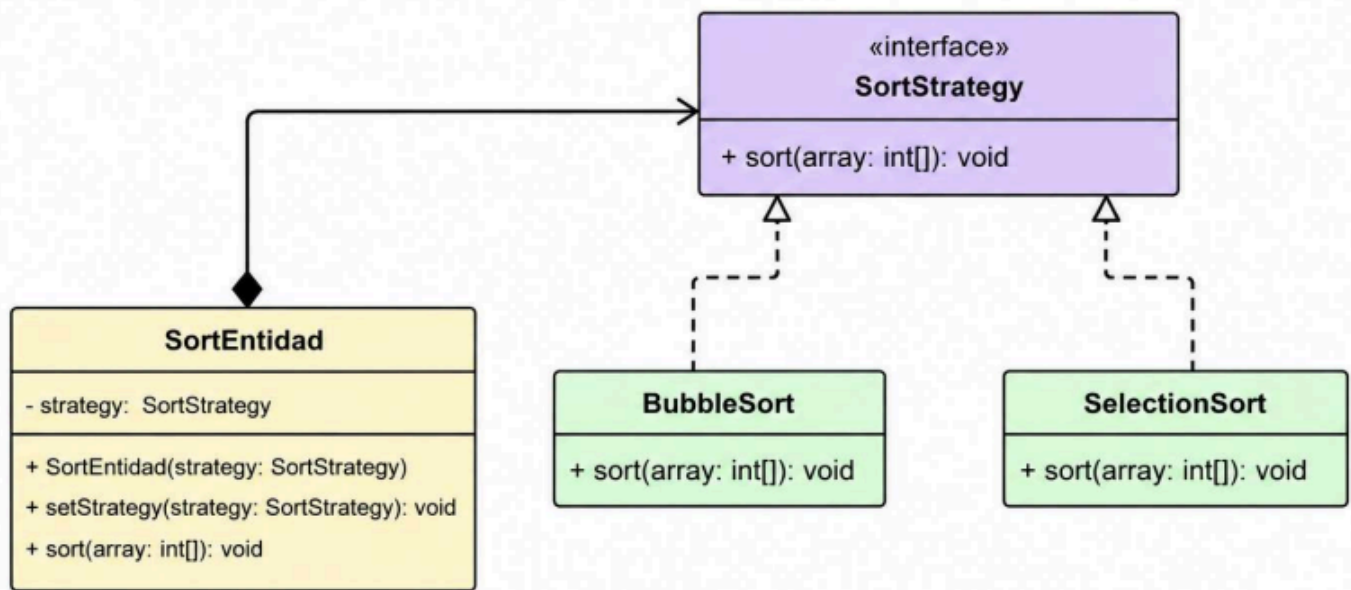
9. Plantilla



10. Ejemplo1



11. Ejemplo2



Leyenda:

- Asociación (usa)
- - - - -> Realización (implementa)
- Contexto
- Estrategias concretas
- Interfaz (Strategy)

El SortEntidad delega el ordenamiento a la estrategia (SortStrategy). Puedes cambiar la estrategia en tiempo de ejecución.